Part II

Phase - 1

Normal Operations - 5 users

Flash Sale - 20 users

What happens to your customers?
5 users: roughly 0.4 RPS, no obvious failures, but latency around 15s
20 users: still roughly 0.4 RPS, no obvious failures, but latency around 60s
Conclusion: the synchronous system is throughput-limited by payment processing, so more users mostly create queueing delay, not more completed orders

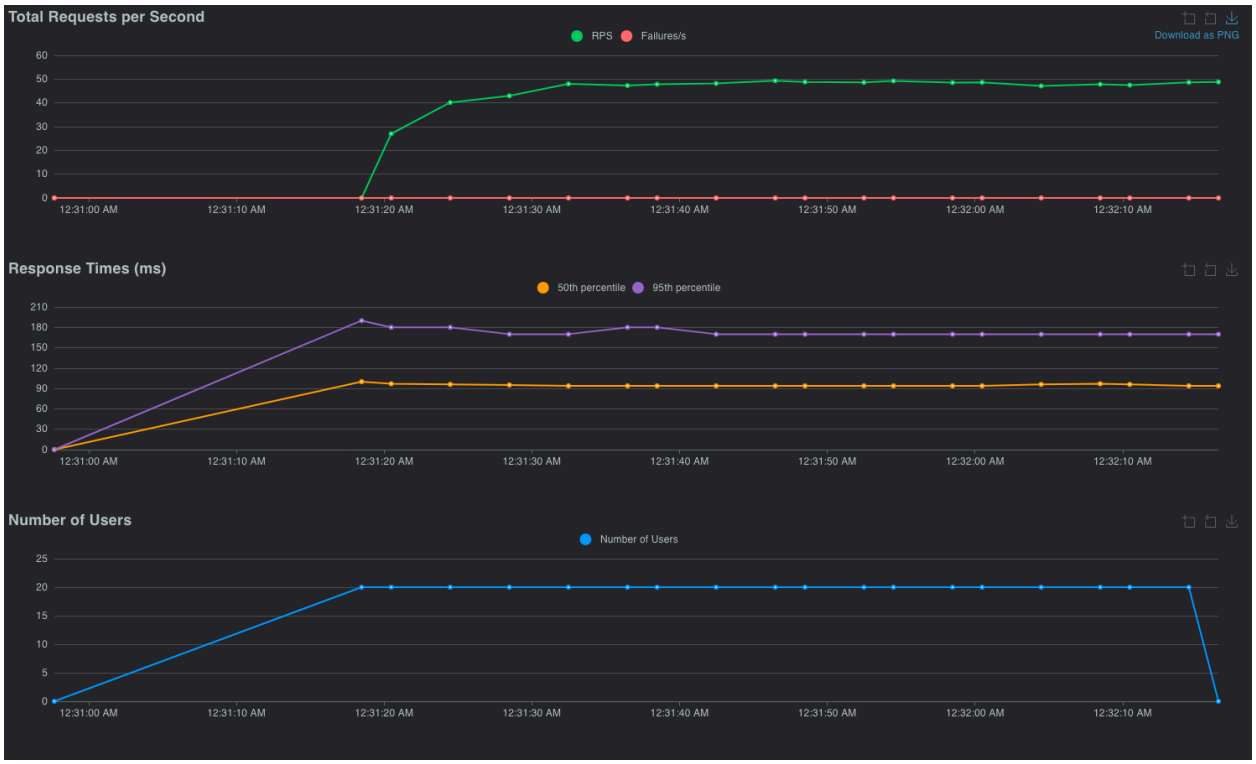
Phase - 2

1. Payment processor speed: 1 order / 3 seconds = 0.33 orders/second
2. With 20 concurrent customers, max throughput is still about 0.33 orders/second
3. Flash sale demand: 20 orders/second
4. Orders lost/backlogged per second: 20 - 0.33 = 19.67 orders/second

The bottleneck is the synchronous payment step. Even with 20 concurrent users, the system can only complete about 0.33 orders per second because each order spends 3 seconds in payment verification and the processor cannot go faster. That means during the flash sale, demand exceeds capacity by about 19.67 orders per second. In practice, more users do not increase throughput; they only increase waiting time, which matches the Phase 1 graphs where response times rose sharply while throughput stayed almost flat.

What can be changed?
Since payment processing cannot be made faster, the request path must change. The fix is to decouple order acceptance from payment processing by moving to an asynchronous architecture: accept the order immediately, publish an event to SNS, queue it in SQS, and process it with background workers.

Phase - 3



In the asynchronous design, the API no longer blocks on payment verification. Instead, it accepts the order immediately, publishes the event to SNS, and returns 202 Accepted. This makes customer-facing latency much lower and allows the system to accept far more orders during the flash sale. The payment bottleneck still exists, but it has been moved behind the queue into the background worker tier.

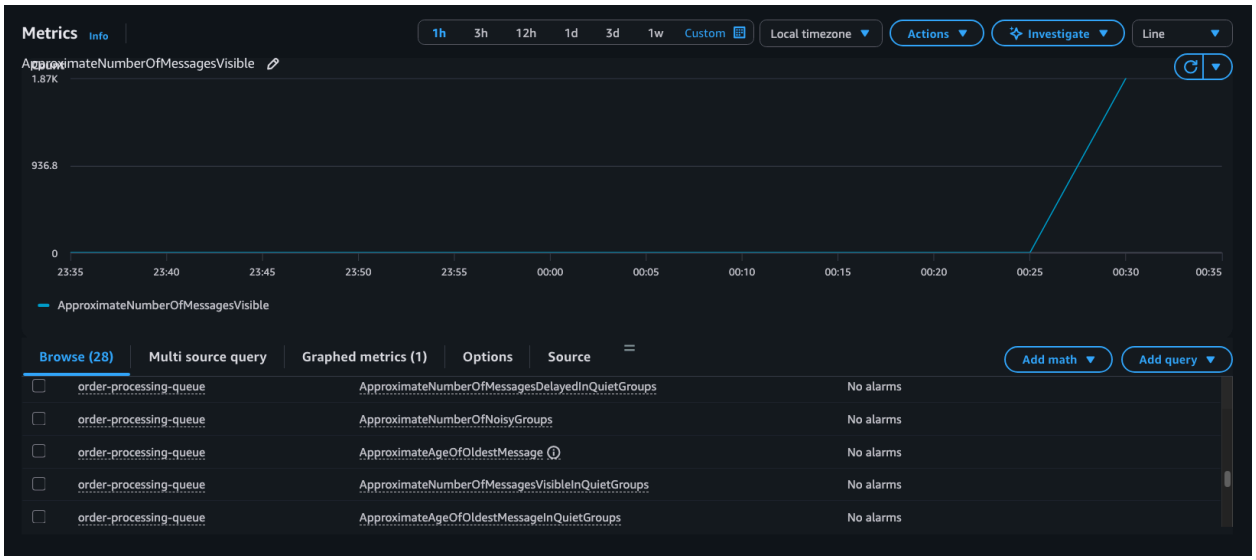
Phase - 4

Queue analysis

Order acceptance rate during async test: about 20 orders/second

Single worker processing rate: 1 order / 3 seconds = 0.33 orders/second

Queue growth rate: 20 - 0.33 = 19.67 messages/second



The CloudWatch screenshot shows the queue rapidly spiking to about 1.87K visible messages, which confirms that orders were accepted much faster than they were processed.

Time to clear backlog
If peak queue depth was about 1,870 messages and one worker processes 0.33 messages/second:

Drain time = 1870 / 0.33 = 5667 seconds
5667 / 60 = about 94.5 minutes

The async architecture solved the customer-facing latency problem, but it exposed a new bottleneck in the background processor. Orders were accepted much faster than a single worker could process them, so the SQS queue grew quickly. Based on the measured queue spike of about 1.87K messages and a processing rate of only 0.33 orders per second, the backlog would take about 95 minutes to fully drain with just one worker.

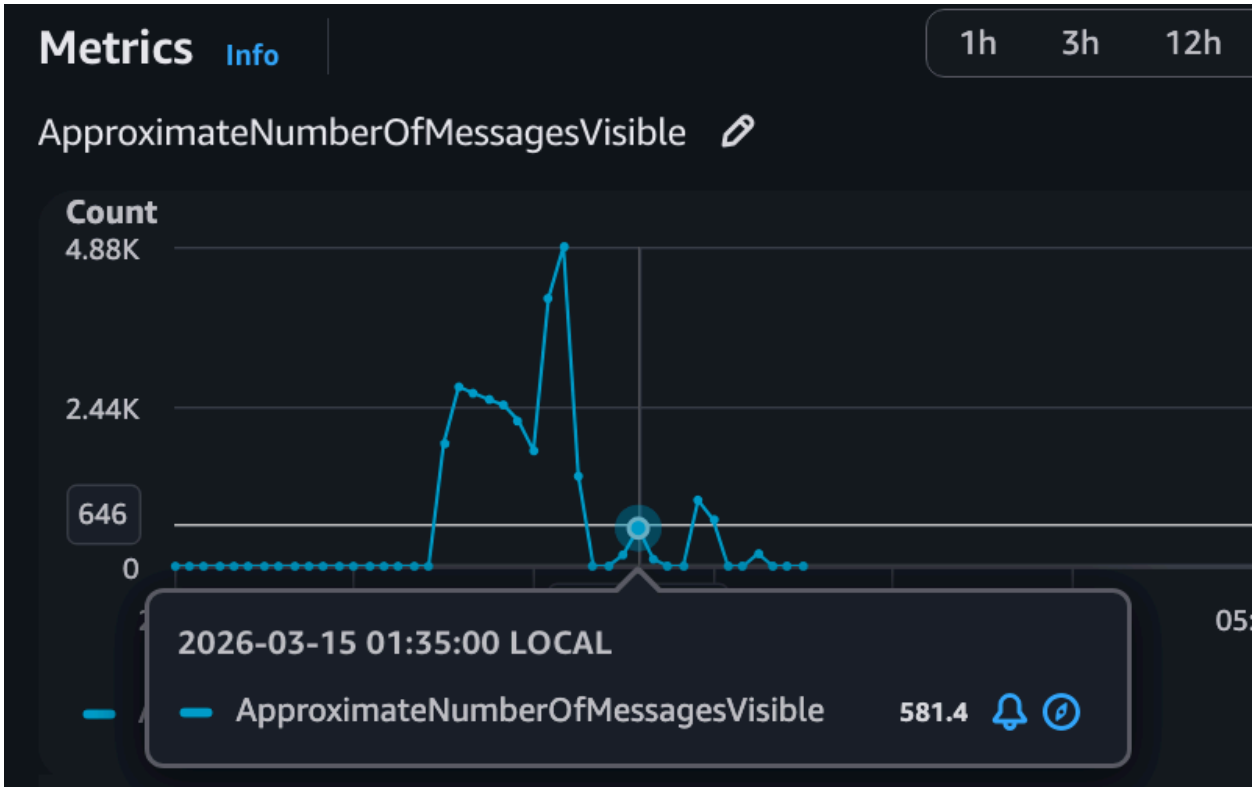
Customers receive immediate acceptance instead of waiting, but actual fulfillment is delayed because orders accumulate in the queue faster than they can be processed.

Phase - 5

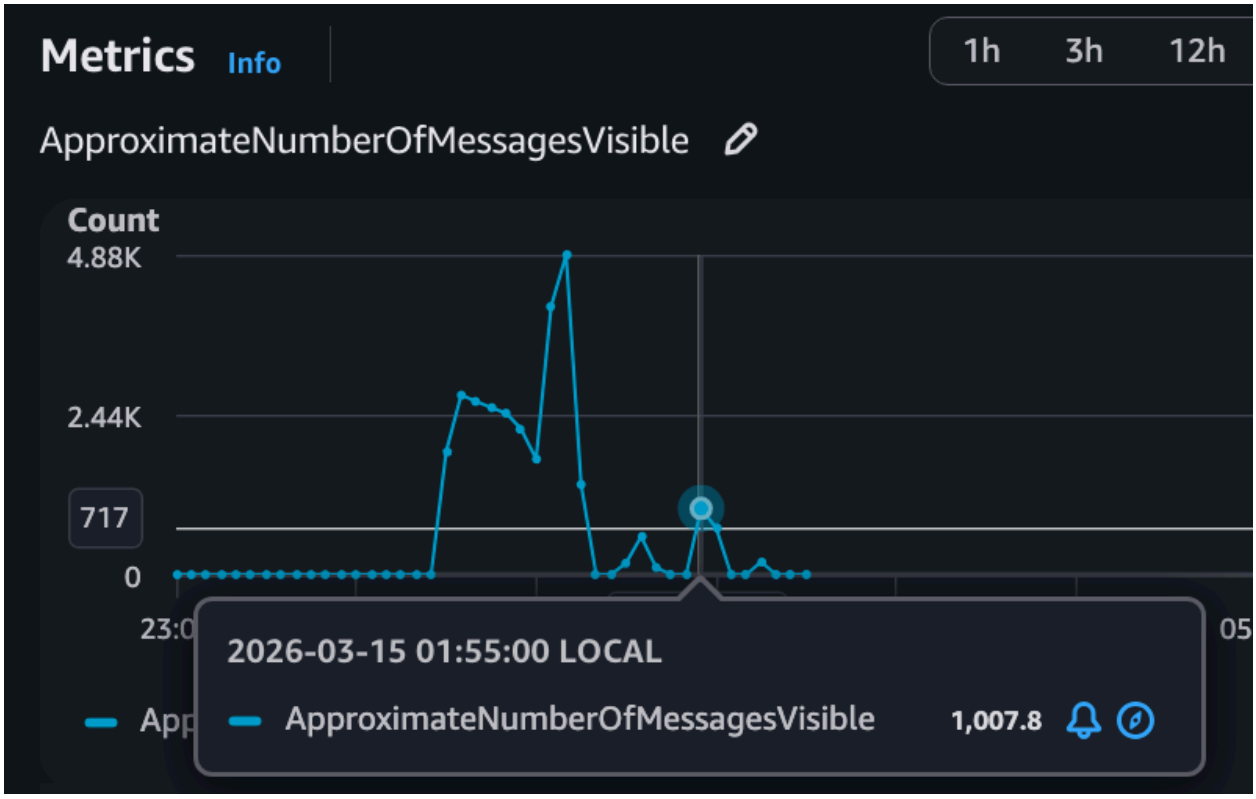
Results

Workers	Processing Rate	Queue Depth	Orders Processed in 60s	Time to Drain
5	1.67/s	581.4	~100	~12 min
20	6.67/s	1007.8	~400	~7 min
100	33.3/s	192	~2000	~3 min

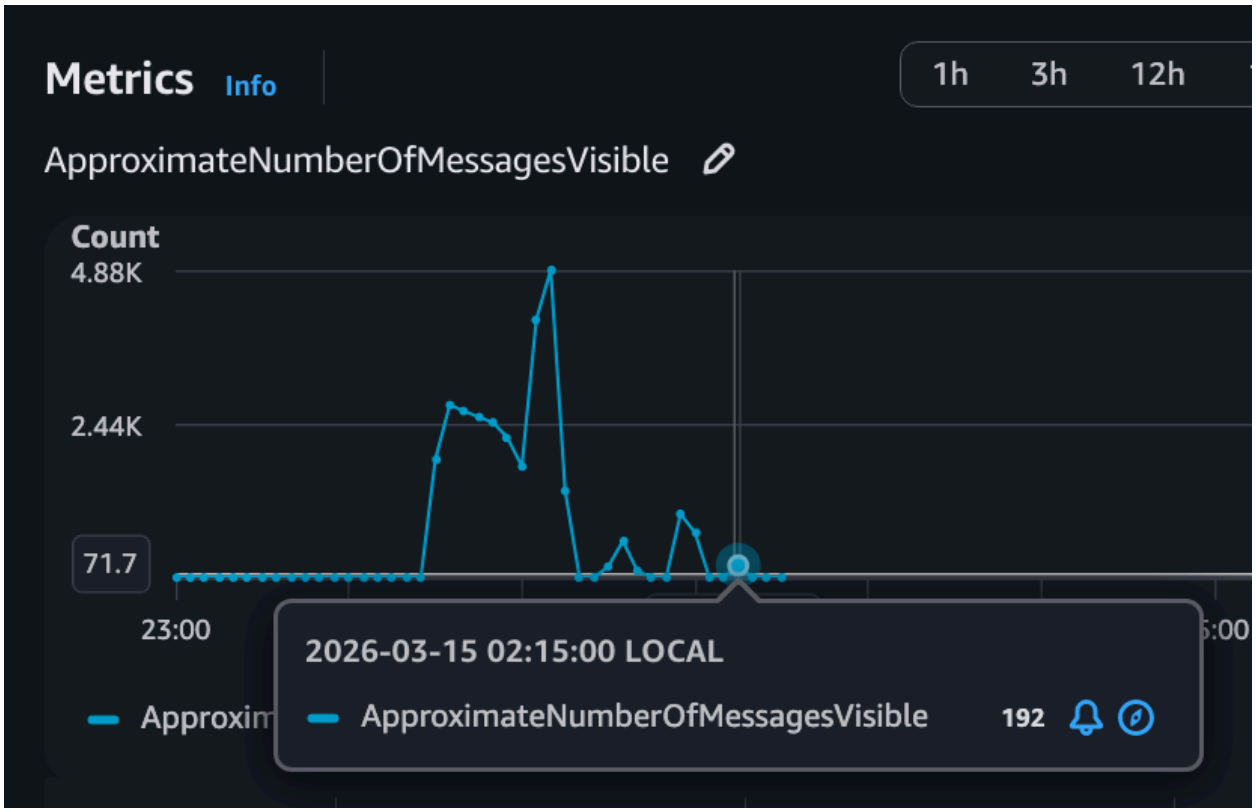
5 workers - still clearly behind the incoming rate. The queue built up and took about 12 minutes to return to zero after the test stopped.



20 workers - much better than 5 workers. The backlog drained faster, reaching zero in about 7 minutes, although queue buildup still occurred during the flash-sale run.


Metrics
Info
1h
3h
12h
ApproximateNumberOfMessagesVisible
Count
4.88K
2.44K
717
0
23:0
05
2026-03-15 01:55:00 LOCAL
ApproximateNumberOfMessagesVisible
1,007.8

100 workers - the best result. Queue buildup was much smaller, and the backlog drained to zero in about 3 minutes after the test ended.



Minimum Workers to Prevent Buildup

At **60 orders/second** incoming with **3 seconds per payment**:
- Minimum workers = 60 x 3 = **180 goroutines**

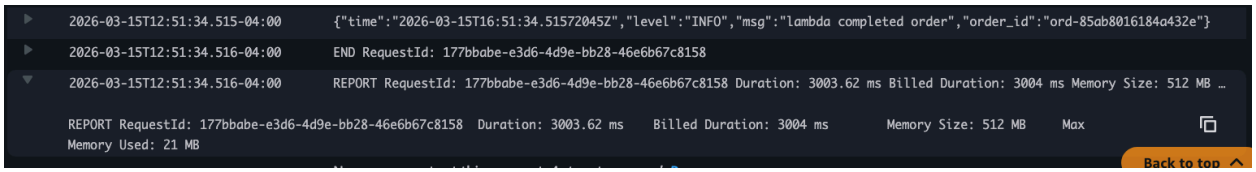
At our measured incoming rate of about **48-50 orders/second**:
- Minimum workers = 48 x 3 = 144 goroutines
- or 50 x 3 = **150 goroutines**

So **100 workers** improved performance a lot, but it was still below the theoretical level needed to guarantee zero backlog under sustained measured load. To fully prevent queue buildup, we would want roughly **145-150 concurrent workers**.

Part III

Cold Start

Warm Start

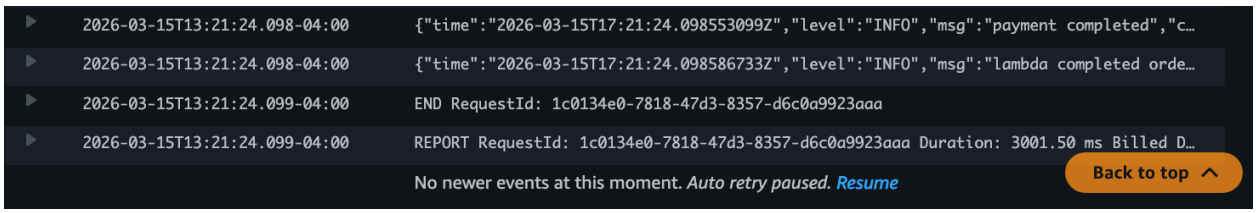


Questions

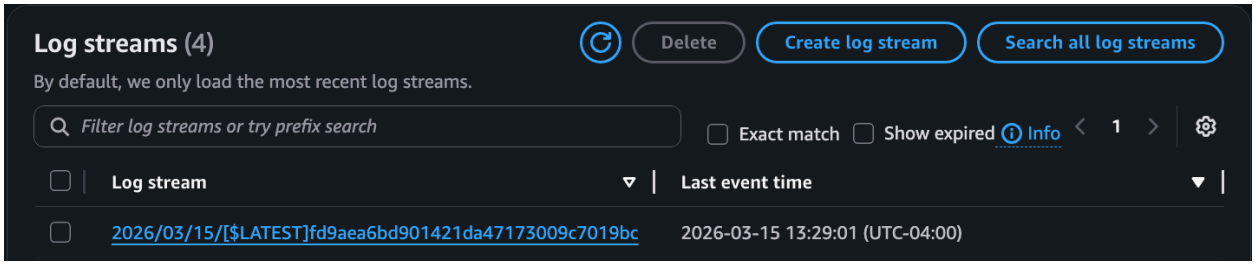
How often do cold starts occur?

Cold starts occurred on the first request and again after about 8 minutes of inactivity.

Sent the last request at 13:21



Then sent the new request at 13:29, tried sending requests before this time but they ended up being warm starts.



What's the overhead?
The cold-start overhead was about 76.96 ms on a roughly 3000 ms execution, which is about 2.6% overhead.

Does this matter for 3-second payment processing?
For a 3-second payment workflow, this overhead is not very significant. It is measurable, but small relative to the total processing time.

Math
Using AWS Lambda's published pricing of **$0.20 / 1M requests**, **$0.0000166667 / GB-second**, and the free tier of **1M requests + 400K GB-seconds** (AWS Lambda pricing):

My **warm result** was **3004 ms** billed at **512 MB**:
3.004 × 0.5 = 1.502 GB-s per request
My cold result was **3083 ms** billed at **512 MB**:
3.083 × 0.5 = 1.5415 GB-s per request

So the cost math is:

10,000 orders/month
Warm: 10,000 × 1.502 = **15,020 GB-s**
Cold: 10,000 × 1.5415 = **15,415 GB-s**
Both are far under **400,000 GB-s**, and **10,000 < 1,000,000** requests, so **monthly Lambda cost = $0**

Free-tier capacity based on your measured results
Warm: 400,000 / 1.502 ≈ **266,311 orders/month free**
Cold: 400,000 / 1.5415 ≈ **259,487 orders/month free**

Break-even vs Part II ECS cost of $17/month
Warm case: about 945K orders/month
Cold-only worst case: about 922K orders/month

Should your startup switch to Lambda? Why or why not?
Our startup should switch to Lambda only if minimizing operational overhead is more important than having strong delivery guarantees. In our testing, cold starts happened on the first request and again after about 8 minutes of inactivity, but the overhead was only about 76.96 ms on a 3-second payment flow, so performance impact was small. The cost advantage is compelling because at 10,000 orders per month Lambda would cost $0, while the Part II ECS setup costs about $17 per month, and Lambda remains effectively free until roughly 260K orders per month based on our measured execution time. However, the trade-off is reliability: unlike the SQS-based design, Lambda processing through SNS does not provide the same queueing guarantees, and failed messages can be retried only a limited number of times before being lost. For a small startup with moderate order volume and tolerance for that trade-off, Lambda is the simpler and cheaper choice; if guaranteed processing and stronger failure handling matter more, the SQS-based architecture is safer.